

Table of Contents

1. Why I Built This
2. Project Setup & Environment
3. Understanding the Five Strategies
4. Building the Aggregator
5. Backtesting Framework
6. First Results — 30 Days
7. Raising the Agreement Threshold to 4/5
8. Improving Individual Strategies
9. Testing Confidence Thresholds (60 / 70 / 75%)
10. Position Sizing & PnL Modelling
11. Multi-Asset Testing
12. The Confidence Table — Reading Each Trade
13. What's Next — The LLM Meta-Filter
14. Key Lessons & Prompt Patterns

HyperLiquid trade: (1W/0L)



Perps + Spot + Vaults ▾	All-time ▾
PNL	\$1,069.48

Proof:

<https://hypurrsan.io/address/0x27152ee1313FcF00C80aa4c8358B574B3d7c29A7#more>

Public trading wallet: 0x27152ee1313FcF00C80aa4c8358B574B3d7c29A7

Even on hyperliquid, I never let AI take full control of my account I only used the model/bot to help me in gaining more confidence on my trades... fully automating or trusting a model with your real money would be really stupid thing to do... (if u DO NOT have any prior experience) so I would highly suggest don't do that unless u are very confident with your bot/strategy or have a proper risk management strategy in place.

Zerodha Trades: (5W/6L) (Indian Market result):

As u can see the winrate is lesser than 50% but because my RR was better than 1:1 I was able to profit but in the long run probably I would have wiped out my entire balance that's why I stopped using it for Indian market

Realised P&L	Charges & taxes	Other credits & debits	Net realised P&L	Unrealised P&L
+10.13L	31.41k 	0	+9.82L	0

Showing page 1 (rows 1 - 11 out of 11)  Last updated: 2026-05-22 Search Download

Symbol 	 Qty.	 Buy avg.	 Buy value	 Sell avg.	 Sell value	Realised P&L	Unrealised P&L
NIFTY2651923350PE	5850	110.20	6,44,650.50	132.12	7,72,918.25	+1,28,267.75 +19.90%	–
NIFTY2651923400CE	9360	110.46	10,33,935.50	196.80	18,42,022.00	+8,08,086.50 +78.16%	–
NIFTY2651923450PE	10270	101.24	10,39,779.00	52.94	5,43,689.25	-4,96,089.75 -47.71%	–
NIFTY2651923650CE	27625	12.29	3,39,488.49	1.46	40,429.99	-2,99,058.50 -88.09%	–
NIFTY2651923650PE	30030	30.00	9,00,900.00	12.23	3,67,204.50	-5,33,695.50 -59.24%	–
NIFTY2651923700CE	17810	61.92	11,02,809.50	58.11	10,34,917.00	-67,892.50 -6.16%	–
NIFTY2651923700PE	22425	42.96	9,63,394.26	64.53	14,47,130.75	+4,83,736.50 +50.21%	–
NIFTY2651923750PE	6500	149.84	9,73,956.75	236.86	15,39,606.25	+5,65,649.50 +58.08%	–
NIFTY2651923800CE	22555	26.22	5,91,337.49	6.58	1,48,349.49	-4,42,988.00 -74.91%	–
SENSEX2651474800PE	11680	134.26	15,68,107.00	255.49	29,84,079.00	+14,15,972.00 +90.30%	–
SENSEX2651475200PE	6920	147.61	10,21,451.00	68.38	4,73,157.00	-5,48,294.00 -53.68%	–

1. Why I Built This

I built this AI trading bot project primarily as an experiment to explore the capabilities of Claude and understand how large language models can assist in structured decision-making. My goal was not to create a fully autonomous trading system, but rather to see whether AI could help improve my own trading process by organizing information, analyzing market conditions, summarizing patterns, and supporting better-informed decisions. Throughout this journey, I treated the project as a learning experience in AI-assisted workflows, prompt engineering, automation, and trading psychology, while always keeping human judgment at the center of every trade.

Disclaimer

“This project was never designed to allow AI to directly control real-money trading accounts, and I strongly discourage anyone from giving autonomous AI systems unrestricted access to execute live trades. Financial markets are highly unpredictable, and relying entirely on AI for trading decisions can be extremely risky and may lead to significant financial losses. In my setup, Claude was only used as a decision-support assistant to help improve analysis and reasoning, while all final trading actions remained under my manual control at every stage of the process.”

What 'using AI as a collaborator' actually means

Most AI-assisted coding looks like: ask AI to write code, copy-paste, move on. That approach works for boilerplate but fails for research tasks where the problem itself is not yet defined. Throughout this project the prompts were more exploratory — asking Claude to reason about trade-offs, question assumptions, and flag risks rather than just generate code.

Every major decision in this document came from a conversation, not a single command. The prompts matter as much as the code.

Software & Tools Required to Build the AI Trading Bot

1. Visual Studio Code (VS Code)

The main development environment used for the project was Visual Studio Code. VS Code made it easy to manage project files, write scripts, test prompts, and organize the overall workflow in a clean and efficient way.

2. Claude AI Extension for VS Code

To integrate AI directly into my development workflow, I used the Claude extension inside VS Code. This allowed me to interact with Claude while coding, testing ideas, debugging scripts, improving prompts, and generating structured trading logic more efficiently.

How to install the Claude extension in VS Code:

1. Open VS Code
2. Go to the **Extensions Marketplace** (Ctrl + Shift + X)
3. Search for **Claude**
4. Install the official Claude/Anthropic-related extension
5. Sign in with your Claude or Anthropic account if required
6. Start using Claude directly inside the editor

From here I will show how to set up a strategy and backtest using hyperliquid since pulling historical data is way easier on hyperliquid than other brokers/(Indian brokers specifically) but at the end of the document I have prompts which can be used for any broker or exchange u just gotta input the name

2. Project Setup & Environment

The project runs on Python 3.11+ on Windows 11. The exchange SDK is the official Hyperliquid Python SDK, which provides a read-only Info client for fetching candle data and a write client for placing orders. All secrets (API keys, wallet addresses) are stored in a .env file and loaded at runtime — never hardcoded.

Core dependencies

Package	Purpose
hyperliquid-python-sdk	Exchange data + order execution
pandas	Candle data manipulation and indicator calculation
numpy	Numerical helpers
python-dotenv	Environment variable loading
pyyaml	Configuration file parsing
python-docx	This document

Project structure

```
hyperbot/  
  strategies/  
    base.py          # StrategySignal dataclass, shared indicators (EMA, ATR,  
MACD, BB)  
    ema_trend.py     # EMA Trend Pullback strategy  
    rsi_meanrev.py   # RSI Mean Reversion strategy  
    bb_squeeze.py    # Bollinger Band Squeeze strategy  
    fvg.py           # Fair Value Gap (SMC) strategy  
    macd_momentum.py # MACD Momentum strategy  
  aggregator.py      # Combines signals, applies thresholds  
  backtest.py        # Walk-forward backtesting engine  
  analyze.py         # Live read-only market analysis  
  pnl_calc.py        # PnL simulation from backtest JSON  
  show_signals.py    # Per-trade confidence table  
  config.yaml        # Symbol, interval, strategy parameters  
  .env              # API keys (gitignored)
```

Initial setup prompt:

"Set up a Python project for a trading bot on Hyperliquid testnet. I want to be able to fetch candle data, run technical strategies, and backtest them. Don't set up live trading yet — just the data pipeline and strategy scaffold."

Note: Starting with testnet only is a critical constraint. It forces the architecture to separate data fetching from order execution cleanly, and removes the risk of accidental live trades during development.

3. Understanding the Five Strategies

Each strategy is a self-contained class with a single `analyze(df)` method that takes a pandas DataFrame of OHLCV candles and returns a `StrategySignal`. The signal contains a `buy_confidence` and `sell_confidence` (each 0–100), a regime label, and a human-readable reason string.

The 0–100 scale is built from four equally-weighted components (0 or 25 points each), summed. This keeps the logic transparent — you can always trace exactly why a strategy scored 75.

3.1 EMA Trend Pullback

Trend-following strategy. Uses a fast EMA (20) and slow EMA (200) to define the macro trend, then waits for price to pull back to the fast EMA before entering in the trend direction. An ATR-based filter prevents entries when price is too far from the EMA (overextended).

Component	Points	What it checks
HTF trend	25	Price above EMA200 (bullish) or below (bearish)
EMA alignment	25	Fast EMA above slow EMA
Pullback zone	25	Price within 0.5 ATR of fast EMA
Candle direction	25	Close > open (bullish bar) or close < open (bearish)

3.2 RSI Mean Reversion

Counter-trend strategy. Looks for RSI extremes (oversold / overbought) combined with price near the 50-period EMA. The logic is that price stretched away from mean tends to revert. This strategy deliberately conflicts with EMA Trend Pullback — that tension is useful.

Component	Points	What it checks
RSI extreme	25	RSI below 30 (buy) or above 70 (sell)
RSI direction	25	RSI moving toward neutral (turning point)
Mean proximity	25	Price within 1.5 ATR of EMA50
Candle direction	25	Candle body confirms the reversal direction

3.3 Bollinger Band Squeeze

Volatility expansion play. Bollinger Bands narrow during low-volatility consolidation (the squeeze). When price breaks out of the bands with volume, a sustained directional move tends to follow. This strategy is regime-aware — it knows whether the market is in squeeze or expansion.

3.4 Fair Value Gap (FVG)

Smart Money Concepts (SMC) strategy. A Fair Value Gap is a three-candle imbalance pattern where the middle candle's body does not overlap with the wicks of the surrounding candles, leaving a price 'gap' that institutions are theorised to revisit. This was the most technically complex strategy to implement correctly.

Note: The FVG strategy required two significant iterations. The first version used wick-based fill detection (marking a gap as filled if subsequent lows touched the gap top), which caused nearly every gap to be marked filled immediately. Switching to close-based fill detection and adding a proximity gate (requiring price to be within 0.25 ATR of the gap) raised the attribution win rate from 31% to 41%.

3.5 MACD Momentum

Classic momentum strategy using MACD line, signal line, and histogram. Detects fresh crossovers (within the last 3 bars), histogram acceleration, and uses EMA200 as a higher-timeframe filter. Works best when combined with trend-following strategies.

Strategy scaffold prompt:

"Build me 5 technical trading strategies as Python classes. Each should have an analyze(df) method returning a signal with buy_confidence and sell_confidence from 0 to 100. Use four 25-point components per strategy so the scoring is transparent. Strategies: EMA trend pullback, RSI mean reversion, Bollinger squeeze, Fair Value Gap, MACD momentum."

4. Building the Aggregator

The aggregator is the core of the system. It runs all five strategies on the same DataFrame, collects their signals, and applies voting logic to produce a single recommendation.

The initial version required 3/5 strategies to have confidence $\geq 50\%$ in the same direction. This was deliberately conservative — requiring multiple independent strategies to agree before firing a signal reduces false positives at the cost of fewer trades.

Aggregator logic

```
agree_buy = count of strategies with buy_confidence >= threshold
agree_sell = count of strategies with sell_confidence >= threshold

if agree_buy >= min_agree and avg_buy >= threshold*0.8 and avg_buy > avg_sell + 15:
    recommendation = 'long'
elif agree_sell >= min_agree and avg_sell >= threshold*0.8 and avg_sell > avg_buy + 15:
    recommendation = 'short'
else:
    recommendation = 'stand Aside'
```

The $\text{avg_buy} > \text{avg_sell} + 15$ condition prevents signals when the direction is ambiguous — even if 4 strategies barely cross the threshold going long, if the average sell confidence is close to the average buy confidence the signal is rejected.

Aggregator prompt:

"Build an aggregator that runs all 5 strategies and combines their signals. Require at least 3 out of 5 to agree (confidence $\geq 50\%$) before recommending long or short. Add a check that the average score in the agreed direction is meaningfully higher than the opposite."

5. Backtesting Framework

The backtester is a walk-forward simulator. It scans historical bars one at a time, feeding only past data to the strategies (no lookahead). When the aggregator fires a signal, it simulates an entry at the close of that candle, then scans forward bar-by-bar to see whether the stop-loss or take-profit is hit first.

Key design decisions

- One trade at a time — no new signal is considered while a trade is open
- Stop = $1.5 \times \text{ATR}(14)$ from entry — dynamic, adapts to current volatility
- Take-profit = $2 \times$ stop distance — fixed 1:2 risk-reward ratio
- Conservative tie-breaking — if both SL and TP are inside the same candle, count as a loss
- Warmup period of 215 bars before the backtest window — ensures EMA200 is properly seeded

Paginated candle fetching

Hyperliquid's API returns approximately 5,000 candles per request. For long backtests (180 days of 15-minute bars = 17,280 bars) the fetch function loops, moving the end timestamp backwards until enough bars are collected.

Backtest extension prompt:

"The backtest needs to support --days parameter for up to 180 days. The API only returns ~5000 candles per call so add pagination. Also show held time per trade and which strategies agreed on each profitable trade."

Note: Always add pagination before running long backtests. A silent truncation (getting 5,000 bars when you asked for 17,000) produces results that look plausible but cover a fraction of the intended period — one of the harder bugs to notice.

6. First Results — 30 Days at 3/5, 50% Confidence

The first meaningful backtest ran on BTC 15m candles for 30 days with the 3/5 agreement threshold at 50% confidence. The result was 128 trades at 31.5% win rate.

Break-even win rate for a 1:2 risk-reward system is 33.3%. At 31.5% the system was losing money, but the result was informative — the strategy attribution table showed which of the five strategies was contributing most to winning trades.

Strategy	Agreed on	Won	Win%
EMA Trend Pullback	128	41	32%
RSI Mean Reversion	115	35	30%
Bollinger Squeeze	112	33	29%
MACD Momentum	108	34	31%
Fair Value Gap	42	13	31%

All five strategies had similar attribution win rates — none stood out as clearly superior or inferior. This was the signal to focus on the aggregation logic rather than the individual strategies.

First results prompt:

"Run backtest for 30 days and show total trades, win rate, held time per trade, and which strategies agreed on profitable trades."

7. Raising the Agreement Threshold to 4/5

The hypothesis: requiring more strategies to agree before trading should filter out marginal setups and raise the win rate, at the cost of fewer trades.

Moving from 3/5 to 4/5 agreement threshold changed the results significantly.

Threshold	Trades (30d)	Win Rate	vs Break-even
3/5 agree	128	31.5%	-1.8%
4/5 agree	69	34.3%	+1.0%

At 4/5, the system crossed the 33.3% break-even line for the first time. Fewer trades but each one carrying more edge. More importantly, splitting by agreement count revealed a dramatic pattern:

Agreement	Trades	Wins	Win Rate
5/5 unanimous	9	6	67%
4/5	98	26	27%

The 5/5 unanimous signals had a 67% win rate — double the 4/5 rate. This was the most important finding in the project: unanimous agreement is qualitatively different from near-unanimous agreement.

Threshold change prompt:

"Make it 4/5 agreement and show the results. Also break down the win rate separately for 4/5 trades vs 5/5 trades."

8. Improving Individual Strategies

The Fair Value Gap strategy had a 31% attribution win rate — the weakest of the five. The goal was to improve it in isolation before changing the aggregator.

What was wrong with FVG v1

- Fill detection used lows — any candle wick touching the gap top marked it as filled
- This caused almost every FVG to be marked filled within 1–2 bars of formation
- The scoring gave 25 points just for 'any FVG exists' regardless of proximity to current price
- No higher-timeframe filter — FVGs formed against the macro trend were equally weighted

FVG v2 changes

- Fill detection uses closing prices — a gap is only filled if price closes inside it
- Proximity gate — scoring requires price to be within 0.25 ATR of the gap edge
- Freshness bonus — gaps older than 15 bars get zero score
- HTF bonus — 25 optional points if price is on the correct side of EMA200

Result: FVG attribution win rate went from 31% to 41% over 30 days. Crucially, the FVG improvement was done in isolation — changing all five strategies simultaneously would have made it impossible to know which change caused which effect.

Strategy improvement prompt:

"Let's improve each strategy one by one. Start with Fair Value Gap. What parameters can we edit? Let's try getting the win rate above 40% for last 30 days."

Note: The attempt to improve all four remaining strategies simultaneously was a failure. Tightening all strategies at the 4/5 threshold left only 9 trades at 22% win rate — individual improvements compound in unexpected ways when requiring near-unanimity. Lesson: improve one strategy at a time and re-run the full backtest before changing another.

9. Testing Confidence Thresholds (60 / 70 / 75%)

The key insight: the 50% threshold for 'a strategy agrees' is arbitrary. A strategy at 51% confidence and a strategy at 99% confidence are treated identically under the original system. Raising the per-strategy threshold filters out weak, barely-crossing signals.

Four thresholds were tested at both 3/5 and 4/5 agreement levels over 30 days on BTC 15m:

Confidence	3/5 — Trades	3/5 — WR	4/5 — Trades	4/5 — WR
50%	131	27.7%	69	39.7% ✓
60%	87	27.9%	29	44.8% ✓
70%	53	26.4%	23	47.8% ✓
75%	46	32.6%	19	52.6% ✓

The 3/5 column shows no meaningful improvement — the agreement count matters more than the confidence threshold when requiring only a bare majority. But the 4/5 column shows consistent improvement: every threshold increase raises the win rate. At 75% @ 4/5, the 30-day result was 52.6% win rate — well above break-even.

Threshold sweep prompt:

"Increase confidence % to 60, 70, and 75 and backtest it for trades when 3/5 agree. Then do the same for 4/5."

Six-month verification at 75% @ 4/5

The 30-day result of 52.6% was on only 19 trades — too small to be reliable. Running the same settings over 180 days produced 30 trades at 40% win rate. The 30-day result was partially a lucky sequence; the 180-day result is more representative.

40% win rate is still above the 33.3% break-even, meaning the system has positive expectancy. However, a losing streak in April (9 consecutive losses) meant that the early drawdown prevented full recovery, resulting in near-flat PnL despite the edge.

Note: Always verify short-period results over a longer window before trusting them. 19 trades is not a large enough sample to distinguish edge from luck.

10. Position Sizing & PnL Modelling

Win rate alone does not determine profitability. Position sizing determines how much of the account is at risk on each trade, and compounding (sizing off current balance) means the sequence of wins and losses matters — not just their ratio.

Scenarios tested (10x leverage, USD 10,000 start)

Scenario	Logic	Result (6 months)
A — Split sizing	4/5 = 10% of balance, 5/5 = 30%	-0.36%
B — 5/5 only at 20%	Only take unanimous signals	+12.99%
C — Flat 20%	All 75%@4/5 trades at 20%	-1.03%
D — Flat 30%	All 75%@4/5 trades at 30%	-2.01%

Scenario B (5/5 unanimous at 20% allocation) produced the best result: +13% over 6 months with only 1.4% maximum drawdown. The 67% win rate of 5/5 trades is genuinely strong — far above break-even — and the small allocation prevents catastrophic drawdowns.

Scenarios C and D show that even at 40% win rate, the compounding effect of an early losing streak can produce a net-negative result over a finite period. Positive expectancy is necessary but not sufficient — you also need enough capital to survive the variance.

Position sizing prompt:

"If we used 10x leverage for each trade, what % return would it have generated if we invested 10% of balance each trade vs increasing size if there is more confidence on the trade? Also run a test where we only take 5/5 trades and use 20% of balance."

11. Multi-Asset Testing

The 5/5 signal rate on BTC 15m was approximately 1–2 per month. To reach 30 signals per month — enough to trade meaningfully — the options were to use a shorter timeframe or run across multiple assets.

Timeframe experiment: 5m and 1m

Running on 5m candles (3x more bars than 15m) across BTC, ETH, and SOL for 30 days produced only 4 five/five trades in total — all of which were losses. The 5m timeframe is too noisy for these strategies. EMA200 on 5m represents only ~16 hours of trend context; on 15m it represents ~50 hours. The signals become less meaningful.

The 1m timeframe was limited by API data availability — Hyperliquid only returns ~5 days of 1m history, making a 30-day backtest impossible.

Timeframe	Assets	5/5 trades / 30d	5/5 WR	Overall WR
15m	BTC only	~1.5/month	67%	34%
15m (6 months)	BTC only	9 in 180d	67%	30%
5m	BTC + ETH + SOL	4 total	0%	26%

Conclusion: the strategies were designed for the 15m timeframe. Compressing to 5m degrades performance across all metrics. To scale trade frequency, run more assets on 15m rather than shortening the timeframe.

Multi-asset / timeframe prompt:

"Drop to a shorter timeframe and run only the 5/5 strategy for 30 days. Drop to such a timeframe that even if 5/5 it takes at least 30 trades."

12. The Confidence Table — Reading Each Trade

One of the most useful additions to the backtest was capturing the per-strategy confidence score for every trade taken. This makes it possible to see not just whether strategies agreed, but by how much — and which combinations tend to produce winners vs losers.

The table below shows the 19 trades taken by the 75% @ 4/5 system over 30 days. An asterisk (*) indicates the strategy met the 75% threshold.

#	Time	Side	Out	EMA	RSI	BB	FVG	MACD
1	2026-04-25 01:15	short	loss	*98%	*75%	*75%	*82%	25%
2	2026-04-25 14:15	short	loss	*75%	*75%	*75%	0%	*75%
3	2026-04-25 16:15	short	loss	*79%	*75%	25%	*76%	*100%
4	2026-04-26 04:45	long	WIN	*78%	*75%	*75%	0%	*100%
5	2026-04-28 09:15	short	WIN	*75%	*75%	*75%	0%	*100%
6	2026-04-30 11:15	short	loss	*75%	*75%	53%	*76%	*100%
7	2026-05-02 02:30	long	loss	*75%	*75%	*75%	0%	*75%
8	2026-05-02 13:00	long	WIN	*75%	*75%	*75%	0%	*100%
9	2026-05-03 11:15	long	WIN	*75%	*75%	51%	*86%	*75%
10	2026-05-03 20:15	long	WIN	*75%	*75%	*96%	0%	*100%
11	2026-05-04 22:00	long	loss	*78%	*75%	*75%	0%	*100%
12	2026-05-06 06:45	long	WIN	*75%	*75%	50%	*79%	*100%
13	2026-05-08 08:00	short	loss	*86%	*75%	*75%	*87%	25%
14	2026-05-10 04:00	long	WIN	*75%	*75%	*75%	0%	*100%
15	2026-05-12 01:15	short	WIN	*75%	*75%	50%	*82%	*75%
16	2026-05-12 23:30	short	loss	*75%	*75%	*75%	0%	*75%
17	2026-05-14 14:15	long	WIN	*75%	*75%	*75%	0%	*100%
18	2026-05-16 17:15	short	WIN	*84%	*75%	*75%	0%	*100%
19	2026-05-19 02:00	short	loss	56%	*75%	*75%	*86%	*75%

Key observations: MACD is the most consistent strategy in this set — it hits 75%+ on almost every trade and tends to score 100% when it agrees. EMA also shows consistently high scores on winning trades. FVG rarely participates but when it does (trades 1, 3, 9, 12, 13, 15, 19), its presence does not reliably predict wins or losses.

Trade 19 is notable: EMA only scored 56% (below threshold) but still fired because the other four strategies hit 75%+. It was a loss. This pattern — where EMA disagrees but the vote still passes — appears in multiple losing trades.

Signal table prompt:

"Show me the confidence % table for each strategy for each of the 19 trades taken in the 75% @ 4/5 setup."

13. What's Next — The LLM Meta-Filter

The mechanical threshold system treats a strategy at 75% exactly the same as one at 100%. It also treats RSI agreeing 'because RSI is at 32 near the oversold extreme' identically to RSI agreeing 'because RSI crossed up from 29 with accelerating momentum.' The number is the same; the quality of the signal is different.

The next step is an LLM meta-filter: a Claude API call that receives the full signal context for a potential trade — every strategy's score, regime, and reason string — and returns a final approve/reject decision with reasoning.

What the LLM can see that the threshold cannot

- Whether strategies are agreeing for the same underlying reason or different reasons
- Regime conflicts — e.g., BB Squeeze saying 'volatility expansion' while MACD says 'momentum stalling'
- Whether the FVG is fresh (just formed) or stale (10 bars old near the fill level)
- EMA disagreement on an otherwise 4/5 setup (as in trade 19)

The hallucination problem

This is the central risk of adding an LLM to a trading system. Unlike the rule-based strategies — which are deterministic and auditable — an LLM can produce confident-sounding reasoning that is factually wrong about the market conditions it was given.

A hallucinated 'this is a strong setup' on a genuinely weak signal, sized at 20-30% of account with 10x leverage, is a significant loss. The model cannot be tested the same way as rule-based logic — you cannot backtest 10,000 LLM decisions cheaply.

Mitigations being considered:

- Use the LLM as a filter only — it can reject but not override a 4/5 mechanical signal
- Require the LLM to cite specific values from the signal data in its reasoning (verifiable)
- Start with paper trading (testnet) and compare LLM-filtered vs unfiltered results over 60+ trades
- Log every LLM decision and reason for post-hoc review

LLM meta-filter prompt:

"What if we make another model which analyses the data — it basically understands why the % of buy/sell is coming from each strategy and then makes a final decision from it, not just blindly agreeing to the recommendation. Does that make sense, will it make a difference?"

14. Key Lessons & Prompt Patterns

Lessons from the build

Lesson	What happened	The fix
Improve one thing at a time	Improving all 4 strategies simultaneously at 4/5 threshold left only 9 trades at 22% WR	Revert all; improve FVG only; re-test before touching others
Agreement count > confidence level	3/5 at 75% gave 32.6% WR; 4/5 at 50% gave 39.7% WR	Prioritise agreement count over per-strategy threshold
Verify short results over long periods	19 trades at 52.6% WR looked great; 30 trades at 40% WR over 6 months is the real number	Always re-run any promising setup over the full available history
Timeframe selection is not neutral	5m signals had 0% 5/5 WR vs 67% on 15m for the same strategies	Strategies are designed for a specific timeframe; don't compress without rebuilding
Sequence risk is real	40% WR system went flat due to early April losing streak	More assets = diversified sequence risk; single-asset sizing must be small
Pagination before long fetches	Silent API truncation returned 5,000 bars when 17,000 were needed	Add pagination loop and log actual bar count before scanning

Prompt patterns that worked well

The isolation prompt

Always scope to one component at a time when debugging or improving.

Isolation pattern:

"Let's improve each strategy one by one. Start with Fair Value Gap only. What parameters can we edit? Let's try getting the win rate above 40%."

The comparison prompt

Ask for multiple variants simultaneously to find the best without sequential guessing.

Comparison pattern:

"Backtest at confidence 60, 70, and 75 for both 3/5 and 4/5 agreement. Show all results in a table."

The honest-question prompt

Asking Claude to explain trade-offs rather than just implement often reveals problems before they happen.

Honest-question pattern:

"Is the confidence % using historical data right now or only telling

its % of buy/sell from the current situation? Which is better to follow?"

The sanity-check prompt

Before acting on results, ask Claude to explain why the numbers make sense — or don't.

Sanity-check pattern:

"The 30-day WR is 52.6% but the 180-day is 40%. Why did it drop and is the 40% number still useful?"

Summary of results

Setup	Period	Trades	Win Rate	PnL (10k, 10x, 20%)
3/5 @ 50%	30 days	128	31.5%	negative
4/5 @ 50%	30 days	69	34.3%	slight positive
5/5 unanimous	180 days	9	67%	+13%
4/5 @ 75%	30 days	19	52.6%	positive
4/5 @ 75%	180 days	30	40.0%	near flat

The most robust finding is the 5/5 unanimity signal: 67% win rate over 6 months is a strong edge. The challenge is frequency — 9 trades in 6 months per asset requires running across many assets simultaneously to generate meaningful volume.

Appendix A — Complete Prompt Reference

This appendix collects every prompt you need to build, test, and deploy your own version of this system — in the order you would use them. Copy these into Claude (claude.ai or Claude Code CLI) and adapt the symbol, timeframe, and parameter values to your own setup.

A1. Build Your Own Trading Bot — Start Here

Use the prompts below in order. Each builds on the previous one. You do not need to run all of them — stop at whatever point matches how far you want to go.

———— Step 1 — Project scaffold ————

This prompt creates the folder structure, installs dependencies, and sets up config files. Use it at the very beginning of a new project.

Step 1 — Scaffold:

```
"I want to build a Python trading bot that connects to [EXCHANGE] and runs technical analysis strategies. Set up the project structure with:  
- A config.yaml for symbol, timeframe, and strategy parameters  
- A .env file for API keys (gitignored)  
- A read-only data client that fetches OHLCV candles  
- A base StrategySignal dataclass with buy_confidence, sell_confidence (0-100), regime, and reason fields  
- Shared indicator functions: EMA, ATR, MACD, Bollinger Bands, RSI  
Do not add live trading yet. I want to build and test strategies before connecting real orders."
```

———— Step 2 — Strategy classes ————

Use this after the scaffold is in place. It creates the five strategy classes. You can use fewer — see Section A2 for individual strategy prompts.

Step 2 — Strategies:

```
"Build 5 technical trading strategies as Python classes, each with analyze(df) returning a StrategySignal. Score each from 0-100 using four 25-point components so the logic is always traceable.  
Strategies:  
1. EMA Trend Pullback — trend-following using EMA20/EMA200 with pullback zone filter  
2. RSI Mean Reversion — counter-trend using RSI extremes near EMA50  
3. Bollinger Band Squeeze — volatility expansion after low-volatility consolidation  
4. Fair Value Gap — three-candle imbalance pattern, close-based fill detection, proximity gate within 0.25 ATR  
5. MACD Momentum — crossover + histogram acceleration + EMA200 HTF filter  
All strategies must handle 'not enough data' gracefully (return 0/0 signal)."
```

———— Step 3 — Aggregator ————

Combines strategy signals into a single recommendation. The key parameters to tune later are agree_threshold (per-strategy minimum confidence) and min_agree (how many must agree).

Step 3 — Aggregator:

```
"Build an aggregator that:
```


- Runs all strategies and collects their StrategySignal outputs
- Counts how many strategies have buy_confidence >= threshold (default 50%)
- Recommends 'long' if min_agree or more strategies agree AND average buy > average sell + 15
- Recommends 'short' symmetrically
- Otherwise returns 'stand_aside'

Make agree_threshold and min_agree configurable parameters with defaults of 50 and 3."

———— Step 4 — Walk-forward backtester ————

This is the core research tool. It simulates trades on historical data without lookahead, one trade at a time. Use it to validate every change before touching live settings.

Step 4 — Backtester:

"Build a walk-forward backtester that:

- Accepts --symbol, --interval, --days, --rr, --confidence, --min-agree as CLI args
- Fetches candles with pagination (API may limit to ~5000 per call)
- Walks bars one at a time, feeding only past data to strategies (no lookahead)
- Enters at close of signal bar with stop = 1.5 x ATR(14), TP = rr x stop
- One trade at a time — no new signal until current trade closes
- Records: time, side, entry, stop, TP, outcome, bars held, strategies agreed, and per-strategy confidence scores
- Prints a trade table and strategy attribution (which strategies agreed on winners)
- Saves full results to backtest_results.json"

———— Step 5 — PnL calculator ————

Run this after each backtest to translate win rates into dollar returns under different position sizing assumptions.

Step 5 — PnL calculator:

"Build pnl_calc.py that reads backtest_results.json and simulates:

- Starting balance: \$10,000, leverage: 10x
- Scenario A: 10% of balance per trade (or 30% if strategies_agreed == 5)
- Scenario B: only take 5/5 unanimous trades at 20% of balance
- Scenario C: flat 20% of balance on all trades

For each scenario print: trades, win rate, final balance, return %, max drawdown, gross wins, gross losses. Use compounding (size off current balance each trade)."

———— Step 6 — Live analysis (read-only) ————

Before connecting live orders, build the read-only analyzer. Run this any time to see what the system currently thinks about a market.

Step 6 — Live analyzer:

"Build analyze.py that fetches the latest N candles for a given

symbol and interval, runs all 5 strategies, and prints a formatted table showing each strategy's buy%, sell%, regime, and reason. Then show the aggregate recommendation with how many strategies agree. This script must NOT place any orders – read-only only."

A2. Individual Strategy Prompts — Pick What Fits You

You do not need all five strategies. Each one represents a different way of reading the market. Choose based on how you think about price action, then use the aggregator with however many strategies you build. Two strong strategies that complement each other outperform five mediocre ones.

———— Strategy 1 — EMA Trend Pullback ————

BEST FOR: Trending markets. This strategy only fires when the macro trend (EMA200) and the short-term trend (EMA20) agree, and price has pulled back to a reasonable entry point. It misses the start of moves but catches the continuation.

AVOID IF: You are trading in choppy, ranging markets with no clear direction. EMA crossovers generate noise when price oscillates around the moving average.

EMA Trend Pullback prompt:

```
"Build an EMA Trend Pullback strategy class with analyze(df)
returning a StrategySignal.
Score using four 25-point components:
1. Price above EMA200 (bullish HTF trend) — 25 pts
2. EMA20 above EMA200 (trend alignment) — 25 pts
3. Price within 0.5 ATR of EMA20 (pullback zone, not overextended) —
25 pts
4. Close > open (bullish candle body confirming entry) — 25 pts
Invert all conditions for the sell score.
Include regime label: 'uptrend', 'downtrend', or 'transitioning'."
```

———— Strategy 2 — RSI Mean Reversion ————

BEST FOR: Range-bound markets. Price oscillates around a mean and RSI extremes (below 30, above 70) signal that a snap-back is likely. This strategy deliberately conflicts with EMA Trend — that conflict is a feature, not a bug. If both agree, the signal is stronger.

AVOID IF: You are in a strong trending market. RSI staying below 30 for extended periods in a downtrend will generate false buy signals repeatedly.

RSI Mean Reversion prompt:

```
"Build an RSI Mean Reversion strategy class with analyze(df)
returning a StrategySignal.
Score using four 25-point components:
1. RSI below oversold threshold (default 30) for buy, above
overbought (70) for sell — 25 pts
2. RSI turning back toward neutral (current RSI > previous RSI for
buy) — 25 pts
3. Price within 1.5 ATR of EMA50 (near mean, not wildly extended) —
25 pts"
```

```
4. Bullish/bearish candle body – 25 pts
Include regime: 'oversold', 'overbought', 'ranging', 'trending'."
```

Strategy 3 — Bollinger Band Squeeze

BEST FOR: Breakout plays after consolidation. When Bollinger Bands narrow to their tightest 30% in a 50-bar window, the market is coiling. A breakout above or below the bands with above-average volume is a high-probability directional move.

AVOID IF: You want to predict breakout direction in advance — this strategy does not. It detects that a breakout is happening, not where it will go. The trade is reactive, not predictive.

Bollinger Squeeze prompt:

```
"Build a Bollinger Band Squeeze strategy class with analyze(df)
returning a StrategySignal.
Score using four 25-point components:
1. Squeeze active: BB width in bottom 30% of its 50-bar range – 25
pts
2. Price above upper band (buy) or below lower band (sell) – 25 pts
3. Volume above 20-bar average (proportional bonus up to 25 pts) – up
to 25 pts
4. Close > open (bullish body) or close < open (bearish body) – 25
pts
Include regime: 'squeeze', 'expansion', 'normal'."
```

Strategy 4 — Fair Value Gap (SMC)

BEST FOR: Traders familiar with Smart Money Concepts (SMC) or institutional order flow. A Fair Value Gap is a three-candle imbalance where a large candle leaves a price region untraded. Price tends to return to fill these gaps before continuing the move.

AVOID IF: You are not comfortable with pattern-based logic or if you are backtesting on a noisy timeframe (under 15m). This strategy has the most implementation complexity and the most parameters to tune.

Fair Value Gap prompt:

```
"Build a Fair Value Gap strategy class with analyze(df) returning a
StrategySignal.
A bullish FVG: candle[i].low > candle[i-2].high (gap between wicks,
middle bar is large).
A bearish FVG: candle[i].high < candle[i-2].low.
Fill detection: use closing prices only (a gap is filled if price
closes inside it – do NOT use wicks, this causes immediate false
fills).
Scoring:
- Proximity: price within 0.25 ATR of gap edge = 35 pts (scale down
linearly past that)
- Freshness: gap formed within last 15 bars = 25 pts bonus"
```

```
- HTF filter: price on correct side of EMA200 = 25 pts bonus  
- Candle confirmation = 15 pts  
Parameters: lookback=50 bars, fresh_bars=15, proximity_atr=0.25,  
min_gap_atr=0.08."
```

———— Strategy 5 — MACD Momentum ————

BEST FOR: Momentum-driven markets where trends develop over multiple sessions. MACD crossovers with accelerating histograms are reliable momentum signals when combined with an EMA200 HTF filter to trade only in the macro direction.

AVOID IF: You are trading in a very slow, low-volatility market. MACD crossovers become frequent and meaningless when price oscillates in a tight range.

MACD Momentum prompt:

```
"Build a MACD Momentum strategy class with analyze(df) returning a  
StrategySignal.  
Use MACD(12, 26, 9) as standard parameters.  
Score using four 25-point components:  
1. Price above EMA200 (HTF bullish filter) - 25 pts  
2. MACD line above signal line - 25 pts  
3. Histogram positive AND larger than previous bar (accelerating  
momentum) - 25 pts  
4. MACD crossed above signal within last 3 bars (fresh signal, not  
stale) - 25 pts  
Invert all for sell score.  
Include regime: 'momentum_up', 'momentum_down', 'neutral'."
```

A3. Going Live — API Integration Steps

This section covers connecting the strategy system to a real exchange for live or paper trading. Follow these steps in order. Do NOT skip the testnet phase — every integration bug that would cost real money will surface there first.

Step-by-step: from backtest to live

1. Create a testnet account and generate an API wallet key from the exchange dashboard. This is a revocable key separate from your main wallet private key.
2. Store all credentials in .env — never in code. Add .env to .gitignore immediately.
3. Build and test the full backtest system on testnet data until results are stable across at least 60+ trades.
4. Build a paper trading mode — the bot calculates and logs what it would do, but does not submit orders. Run this for 2-4 weeks and compare to backtest expectations.
5. Only after paper trading validates backtest results: enable live order submission on testnet with tiny position sizes.
6. Add position size limits, max daily loss limits, and a kill switch (environment variable or file flag to halt trading) before moving to mainnet.
7. Move to mainnet with the smallest viable position size. Scale up only after 30+ live trades match expected win rate.

———— Prompt 1 — Exchange client setup ————

Client setup:

```
"Build a client module for [EXCHANGE] that:  
- Loads API credentials from .env using python-dotenv  
- Provides a read-only function get_candles(symbol, interval, n_bars)  
  that paginates if needed  
- Provides a place_order(symbol, side, size, order_type) function  
  that works on testnet when HL_NETWORK=testnet is set in .env  
- Never touches mainnet unless HL_NETWORK=mainnet is explicitly set  
- Raises a clear error if credentials are missing rather than  
  silently failing"
```

———— Prompt 2 — Live trading loop ————

Use this only after your analysis and backtest code is fully tested. The loop polls the aggregator on a schedule and submits orders when a signal fires.

Live loop:

```
"Build a live trading loop in main.py that:  
- Runs every [interval] seconds (match your candle timeframe)  
- Fetches the latest N candles, runs the aggregator  
- If recommendation is 'long' or 'short' AND no position is currently  
  open:  
  - Calculate entry price (current close), stop (1.5 x ATR), TP (2 x
```

```
stop dist)
- Position size: [X]% of available balance at [leverage]x
- Submit a market order, then immediately submit stop-loss and
take-profit orders
- Log every signal (fired and not fired) with timestamp, scores, and
outcome
- Honour a HALT environment variable – if HALT=1 skip all order
submission
- Never open a second position while one is already open"
```

———— Prompt 3 — Risk controls ————

Add this before going live. Risk controls are not optional — a bug in signal logic without a circuit breaker can drain an account in hours.

Risk controls:

```
"Add the following risk controls to the live trading loop:
1. Max position size: hard cap at [X]% of balance regardless of
confidence
2. Daily loss limit: if total PnL today drops below -[Y]%, stop
trading until midnight
3. Max concurrent trades: only 1 open position at a time
4. Stale signal filter: if the signal bar is more than 2 bars old by
the time the order fires, skip it
5. Minimum ATR: if ATR is below [Z] (market too quiet), skip the
signal
Log every risk control trigger with reason."
```

———— Prompt 4 — Alerts and monitoring ————

Optional but strongly recommended. Running a bot without alerts means you find out about problems from your balance, not from a notification.

Alerts:

```
"Add a notification layer that sends a message via [Telegram /
Discord webhook / email] when:
- A trade is opened (include: symbol, side, entry, stop, TP, size,
which strategies agreed)
- A trade is closed (include: outcome, PnL, duration held)
- A risk control triggers (daily loss limit, halt flag, stale signal)
- The bot restarts or loses connection to the exchange
Keep messages short – one line per event with the key numbers."
```

———— Prompt 5 — The LLM meta-filter (advanced) ————

Add this after the base system is working and validated. This layer calls the Anthropic API to reason about whether a mechanical signal is worth taking.

LLM meta-filter:

```
"Before submitting a live order, call the Claude API with the
following context:
- Current recommendation and how many strategies agree
- Each strategy's buy%, sell%, regime, and reason string
- The proposed entry, stop, and TP"
```

- The last 3 trade outcomes (win/loss) from the log

Ask Claude to return a JSON object: {approve: true/false, confidence: 'high'/'medium'/'low', reason: '...'}

Only proceed with the trade if approve is true AND confidence is 'high'.
Log the full LLM response for every trade (approved and rejected) so you can audit it.
Important: the LLM can only REJECT a mechanical signal – it cannot generate a signal on its own. This prevents hallucinated entries."

Credentials checklist before going live

Item	Where to get it	Where it goes
Exchange API key	Exchange dashboard > API settings	.env — HL_API_KEY
API wallet private key	Generated when creating API key — save once	.env — HL_API_WALLET_KEY
Network flag	Set manually	.env — HL_NETWORK=testnet
Anthropic API key	console.anthropic.com	.env — ANTHROPIC_API_KEY
Telegram bot token	BotFather on Telegram	.env — TG_TOKEN
Telegram chat ID	Send a message to your bot, check getUpdates	.env — TG_CHAT_ID

One final note: the API wallet key is not your main wallet private key. It is a separate, revocable signing key with limited permissions. Never generate it from your main wallet seed phrase. Revoke and regenerate it from the exchange dashboard if you suspect it has been exposed.

End of document.